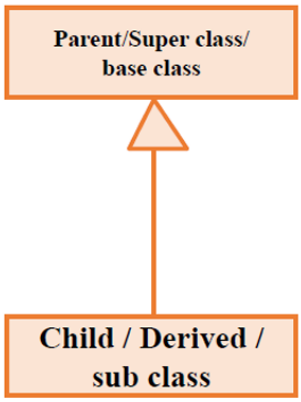
OO-Relationships

In UML, object interconnections (logical or physical), are modelled as relationships.

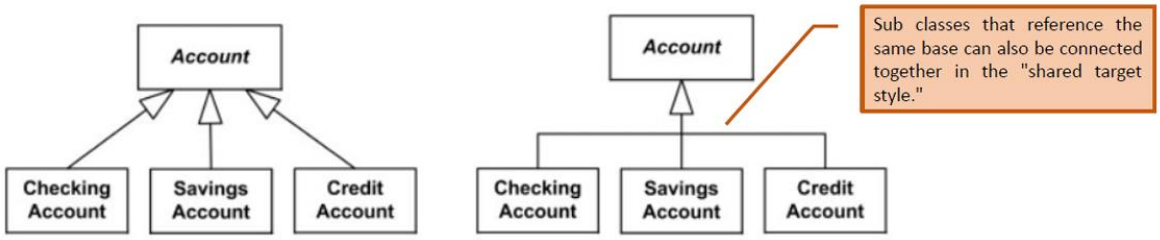
The type of relationships is listed below:
• Generalization- an inheritance relationship
• Abstraction
• Realization - interface implementation
• Dependency
• Association – using relationship
  - Aggregation
  - Composition

Generalization (Inheritance)

- Generalization is **also known as inheritance**
- It represents an **"is-a" or "is-a-kind-of"** relationship
- The **child class is a type of the parent (super) class**
- Used to **show reuse of common elements** in a class diagram
- Child classes **inherit common attributes and methods** from the parent class
- Helps in **code reuse and hierarchical classification**
- Represented by a **solid line with a hollow triangle arrowhead**
- The **arrowhead points to the superclass**
- UML permits a class to inherit from multiple super classes, although some programming languages (e.g., Java) do not permit multiple inheritance.

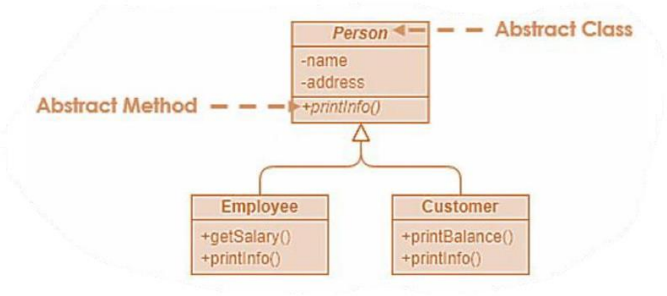


Example:



Abstraction

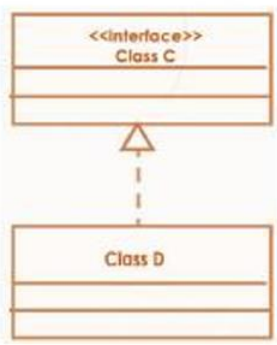
- Abstraction is used in an inheritance hierarchy whereas the parent class defines the framework, and the subclasses implement the specific details.
- The parent class acts as a template that specifies common methods to be implemented by its subclasses.
- An abstract class:
  - Cannot be instantiated directly
  - Contains one or more abstract methods
  - The name of an abstract class is: Typically shown in italics, or
  - Represented using the stereotype {abstract} after or below the class name
- An abstract method:
  - Is a method without implementation
  - Is represented by writing the operation name in italics

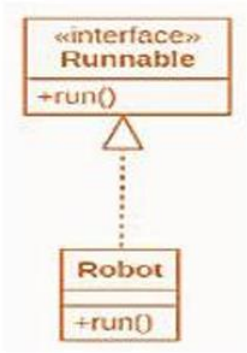


Realization / Interfaces

- Realization is a **specialized abstraction relationship** between two elements
- In this relationship, an **interface specifies a contract**
- A **class guarantees to carry out the contract** by implementing the operations defined in the interface

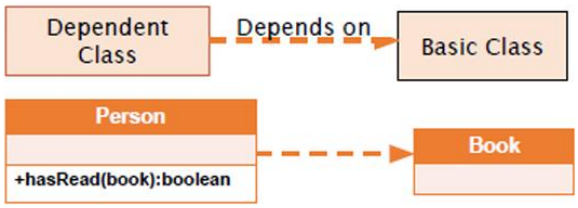
- An **interface** defines a **set of functionalities** without providing implementation
- The implementing class **realizes** the interface by providing concrete implementations of its operations
- Realization represents a **"implements" relationship**
- It is shown using a **dashed directed line with an open arrowhead**
- The **arrowhead points to the interface**





Dependency

- Dependency represents a **"uses" relationship** between two classes
- It indicates that **one class depends on another** for its operation
- A **change in the supplier class** may affect the **dependent (client) class**
- The reverse is **not necessarily true**
- Dependency commonly occurs when:
  - A method **uses an object of another class as a parameter**, or
  - A class temporarily creates or references another class
- Dependency is typically **weaker than association**
- It is shown using a **dashed arrow** pointing from the dependent class to the class it depends on.



An object of one class might use an object of another class in the code of a method. If the object is not stored in any field, then this is modeled as a dependency relationship. For example, the Person class might have a hasRead method with Book as a parameter that returns true if the person has read the book.

Association Relationships

- An association represents a **communication link** between two classes
- If two classes in a model need to **interact or communicate**, an association must exist between them
- Association denotes a **structural relationship** between classes
- The association relationship can be **constrained using navigability**

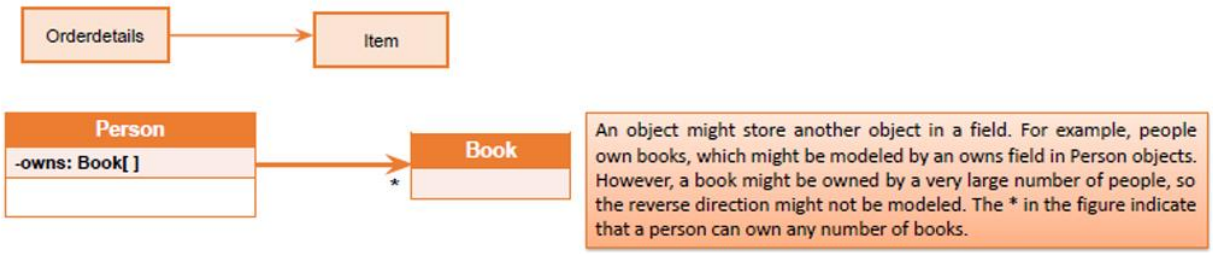
➢ **Types of Association:**

- Unidirectional Association
- Bidirectional Association

Unidirectional Association

- Two classes are related, but **only one class is aware** of the relationship
- One class **knows** about the other, but not vice versa
- Represented by a **solid line with an open arrowhead**
- The **arrowhead points to the known class**
- Includes:
  - **Role name**, and
  - **Multiplicity value**, but only for the known class

- Example:
  - OrderDetails knows about Item, but Item does not know about OrderDetails



Bidirectional Association

- In a **bidirectional association**, two classes are related and **both classes are aware of the relationship**.
- Each class maintains knowledge of the other and can navigate to the other.
- The relationship is **mutual**, meaning objects of both classes can access each other.
- Bidirectional associations are commonly used when **two-way communication or navigation** is required.

Multiplicity in Association

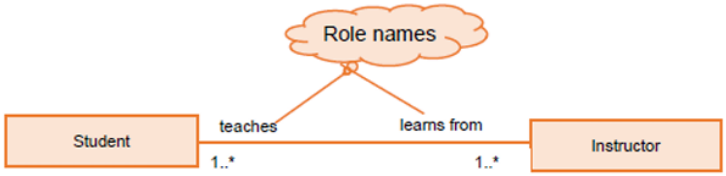
- Multiplicity is used to indicate the **number of possible instances** of one class that can be associated with a **single instance** of another class
- Multiplicity is shown by adding **multiplicity adornments** near the ends of the association line
- Multiplicities can be:
  - **Single values**, or
  - **Ranges of values**

Multiplicities	Meaning
0..1	zero or one instance.
0..* *or* *	no limit on the number of instances (including none).
1	exactly one instance
1..*	at least one instance
n..m	*n* to *m* instances (n and m stand for numbers, e.g. 0..4, 3..15)
n	exactly n instance (where n stands for a number, e.g. 3)

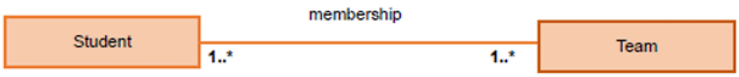
Student has one or more Instructors.

Every Instructor has one or more Students

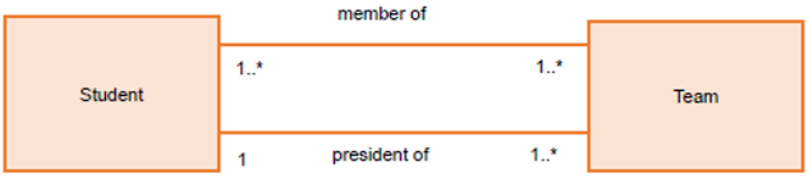
We can also indicate the behaviour of an object in an association (i.e., the role of an object) using role names.

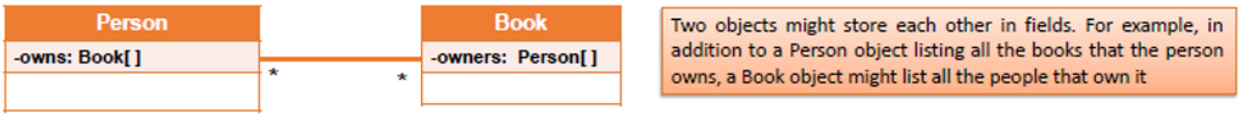


We can also name the association.



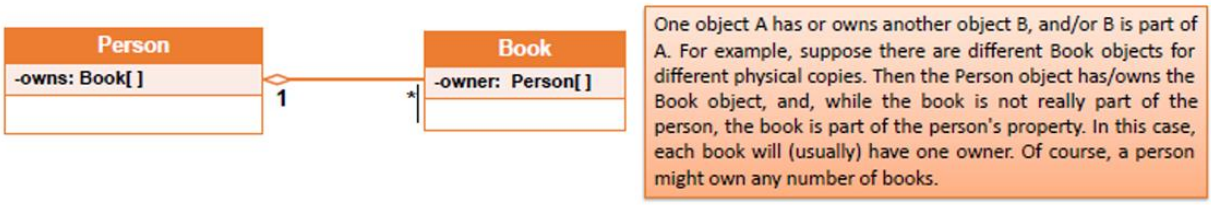
We can specify dual associations using bidirectional association.





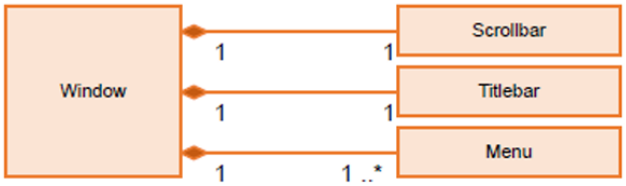
Aggregation

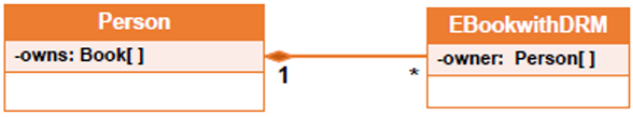
- Aggregation is a **special type of association** used to model objects that **contain other objects**
- It represents a **"has-a" relationship**
- Aggregation defines a **whole–part relationship** between:
  - An **aggregate (whole)**, and
  - A **constituent part**
- In aggregation, the **part can exist independently** of the whole
- Destroying the whole **does not destroy** the part
- Aggregation is denoted by a **hollow diamond (◇)** at the end of the association
- The **diamond is placed on the side of the whole (aggregate)**



Composition

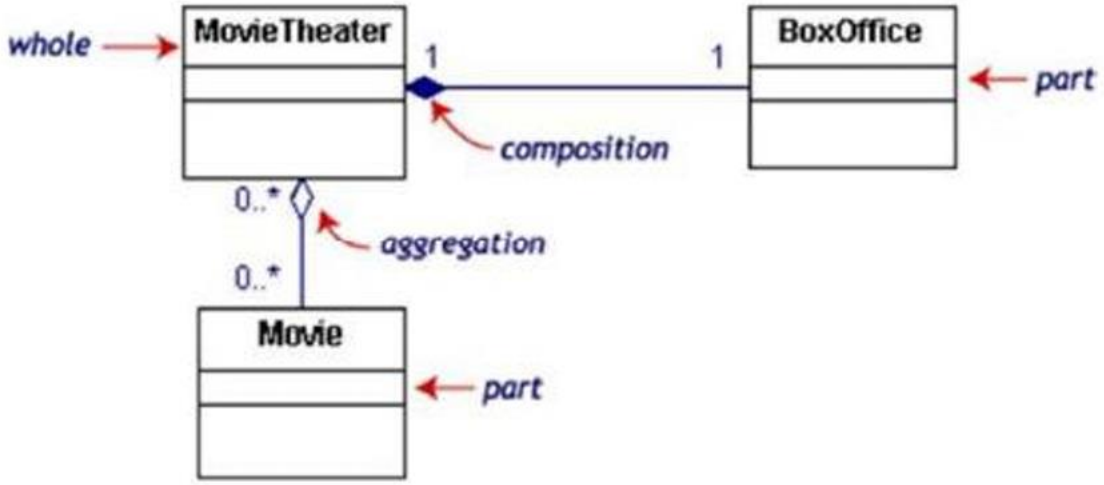
- A composition indicates a strong ownership and coincident lifetime of parts by the whole (i.e., they live and die as a whole).
- Compositions are denoted by a filled-diamond adornment on the association.





In addition to an aggregation relationship, the lifetimes of the objects might be identical, or near For example, in an idealized world of electronic books with DRM (Digital Rights Management), a person can own an ebook, but cannot sell it. After the person dies, no one else can access the ebook. [This is idealized, but might be considered less than ideal.]

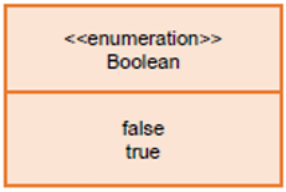
Composition/ Aggregation Example



If the movie theater goes away
so does the box office => composition
but movies may still exist => aggregation

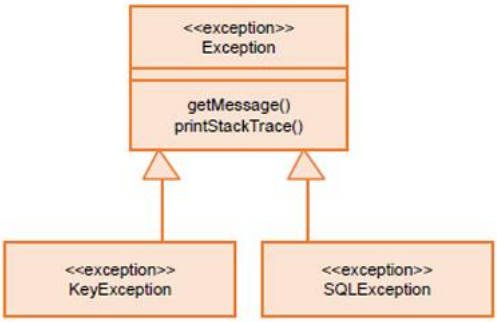
Enumeration

- An enumeration is a user-defined data type that consists of a name and an ordered list of enumeration literals.



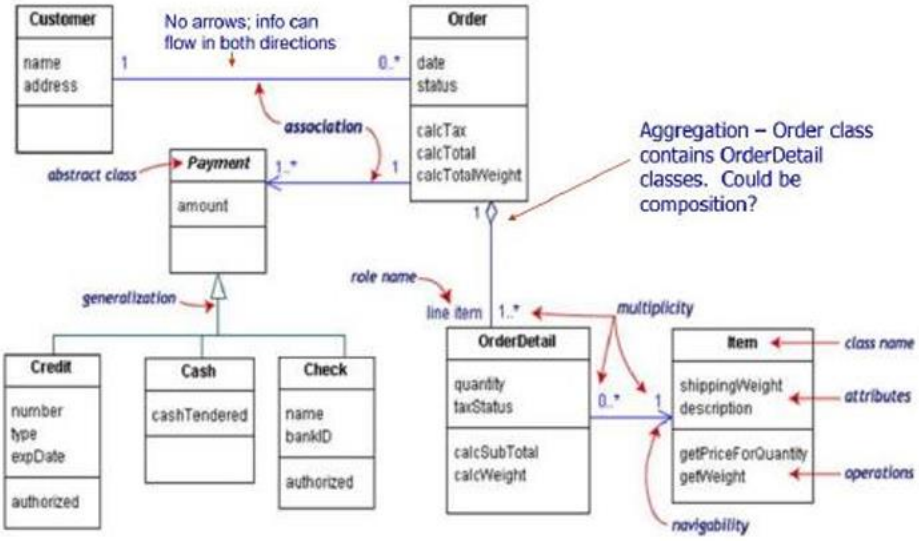
Exceptions

- Exceptions can be modelled just like any other class.
- Notice the <<exception>> stereotype in the name compartment.



How to make a class diagram?

1. Identify all the classes participating in the software solution.
2. Draw them in a class diagram.
3. Identify the attributes.
4. Identify the methods.
5. Add associations, generalizations, aggregations, and dependencies.
6. Add other stuff (roles, constraints, ...)


Customer
name
address
No arrows; info can flow in both directions
1
0..*
Order
date
status
calcTax
calcTotal
calcTotalWeight
association
abstract class
Payment
amount
1..*
1
Aggregation – Order class contains OrderDetail classes. Could be composition?
1
role name
generalization
line item
1..*
multiplicity
Credit
number
type
expDate
authorized
Cash
cashTendered
Check
name
bankID
authorized
OrderDetail
quantity
taxStatus
calcSubTotal
calcWeight
0..*
1
Item
shippingWeight
description
getPriceForQuantity
getWeight
class name
attributes
operations
navigability